

Continuous scheduled time and continuous NPV

no periods, no capacity sigmoids, and one irreducible rank

20 August 2026

Settled.

1. F is the *objective*, not a second encoder pass. The chain is $x \rightarrow s^* \rightarrow t \rightarrow \text{NPV}(x, t)$, differentiated end to end.
2. One resource only: mining / strip capacity. Processing is out of scope.
3. Time stays continuous. No integer periods, no period one-hot, no horizon gate unless we later decide blocks past the horizon are abandoned rather than merely discounted (§6).

1. Two different “logistic” things, only one of which is avoidable

I conflated them in the last note and you were right to push back.

The period machinery — $\lceil t \rceil$, the soft one-hot π_{jk} , the capacity-crossing stack $p_r[i] = \sum_t \sigma((W_r[i] - C_{r,t})/\tau_w)$ in `soft_npv`, the horizon gate $\sigma((T - p)/\tau_h)$, and the straddling-block split in `capacity_cuts`. **All of this is avoidable and all of it goes** (§3). It exists for exactly one reason: to answer “which integer period does this cumulative tonnage land in?” Once time is continuous the question is not asked.

The ordering indicator $\mathbf{1}[s_i^* > s_j^*]$. This is *not* avoidable, because “mine in score order” is a rank and a rank is discontinuous in the scores (§5). What *is* free is the choice of how to smooth it — and the logistic is the wrong choice here. It was inherited from the existing `soft_npv` code, not chosen on merit.

2. The occupancy fraction and the time map

Single resource. Let w_j be block j ’s tonnage and C the mining capacity in tonnes per period. Then

$$\tau_j = \frac{w_j}{C} \quad (\text{periods}).$$

With $\sigma_j := G(s_j^*)$ the *start* time and $t_j = \sigma_j + \tau_j$ the *completion* time,

$$\sigma_j = G(s_j^*) = \sum_i \tau_i \mathbf{1}[s_i^* > s_j^*] = \frac{1}{C} \sum_i w_i \mathbf{1}[s_i^* > s_j^*]$$

i.e. σ_j is just the tonnage that outranks j , divided by the capacity rate. That is the entire construction. It is a pure unit conversion from cumulative tonnage to time, and Proposition 1 of the previous note survives unchanged: G is non-increasing, so $s_i^* \geq s_j^*$ on a precedence edge $\Rightarrow \sigma_i \leq \sigma_j$, and precedence in score space is inherited by time at no cost.

Remark 1 (time-varying capacity, still no sigmoids). If capacity is C_t rather than constant, let $\hat{C}$ be the cumulative-capacity curve, $\hat{C}(u) = \int_0^u C(v) dv$, and let $W_j = \sum_i w_i \mathbf{1}[s_i^* > s_j^*]$ be cumulative tonnage. Then $\sigma_j = \hat{C}^{-1}(W_j)$. Since $\hat{C}$ is strictly increasing and piecewise linear, $\hat{C}^{-1}$ is available in closed form and is differentiable almost everywhere with $(\hat{C}^{-1})'(W) = 1/C_{t(W)}$. Still exact, still no relaxation.

3. What the period machinery was for, and why it can go

In the discrete formulation the cumulative tonnage axis and the period axis are different objects that must be reconciled: a block generally straddles a period boundary, so `capacity_cuts` splits it pro rata and `soft_npv` locates it with a sigmoid stack across T thresholds. In continuous time the two axes are the *same* axis, rescaled by C . There is no boundary to straddle, nothing to split, and nothing to locate.

Concretely, dropping periods removes:

- the $O(nT)$ threshold stack, replaced by a division by C ;
- the temperature τ_w and its “2% of mean cap” heuristic;
- the temperature τ_h and the horizon gate;
- the pro-rata straddle split and the tonnage-weighted overlap discount;
- the associated relaxation error, which currently sits between the surrogate NPV and the hard NPV and has to be reported as a gap.

That is one hyperparameter left (h , the ordering bandwidth) where there were three, and one source of surrogate bias where there were two. This is a strict simplification, not a trade.

4. Continuous NPV

Let r be the discount rate per period and $\delta = \ln(1 + r)$, so the per-period factor $\gamma = (1 + r)^{-1}$ satisfies $\gamma^u = e^{-\delta u}$. A block is not mined at an instant; it is mined over the interval $[\sigma_j, \sigma_j + \tau_j]$. Charging its value at a uniform rate v_j/τ_j over that interval,

$$\text{NPV}_j = \int_{\sigma_j}^{\sigma_j + \tau_j} \frac{v_j}{\tau_j} e^{-\delta u} du = v_j e^{-\delta \sigma_j} \cdot \underbrace{\frac{1 - e^{-\delta \tau_j}}{\delta \tau_j}}_{\psi(\tau_j)},$$

so

$$\boxed{\text{NPV} = \sum_j v_j \psi(\tau_j) e^{-\delta G(s_j^*)}}$$

Three things to notice.

ψ is a constant. It depends only on τ_j , which is fixed data (w_j/C), never on the scores. So it is a one-off per-block reweighting computed once. It satisfies $\psi(\tau) \in (0, 1]$, $\psi(\tau) \rightarrow 1$ as $\tau \rightarrow 0$, and it is exactly the continuum limit of the tonnage-weighted overlap discount that `capacity_cuts` computes by splitting blocks. Same quantity, closed form, no splitting.

All the coupling lives in G . The objective is a smooth, separable function of the vector $(G(s_1^*), \dots, G(s_n^*))$; every block-to-block interaction is inside G . Hence

$$\frac{\partial \text{NPV}}{\partial s_k^*} = -\delta \sum_j v_j \psi_j e^{-\delta \sigma_j} \frac{\partial \sigma_j}{\partial s_k^*},$$

and only $\partial G/\partial s^*$ needs any care at all.

Negative blocks behave correctly. For waste, $v_j < 0$ and $e^{-\delta\sigma_j}$ decreasing means deferring it *increases* NPV. That is the right economics and it comes out of the formula for free, with no special casing.

5. The one irreducible non-smoothness — and why not the logistic

G contains $\mathbf{1}[s_i^* > s_j^*]$, which is a rank. Ranks are piecewise constant in the scores, so the exact G has zero gradient almost everywhere. There are three honest positions.

(A) Keep it hard. If the outer optimiser is Bayesian optimisation over scores or targets — `target_bo`, `policy_bo`, `solution_bo` — no gradient is needed and smoothing is pure loss. Sort by score, take a cumulative sum of w , divide by C : exact, $O(n \log n)$, zero hyperparameters, zero surrogate gap. **If the loop is BO, use this and stop reading this section.**

(B) Smooth it, but not with a logistic. Replace the indicator with a kernel CDF K of bandwidth h :

$$G_h(s) = \sum_i \tau_i K\left(\frac{s_i^* - s}{h}\right).$$

The logistic never saturates. A block ranked far *below* j still contributes $\tau_i \sigma(-\Delta_i/h) > 0$, and a block far *above* contributes $\tau_i(1 - \sigma(\Delta_i/h))$ short of its full τ_i . Individually these are $O(e^{-\Delta/h})$, but they are summed over n blocks and they do not cancel, so G_h carries a systematic bias that grows with n — precisely the regime we are in.

A **compactly supported** kernel removes this exactly. Epanechnikov, for instance:

$$K(u) = \begin{cases} 0, & u \leq -1, \\ \frac{1}{2} + \frac{3}{4}u - \frac{1}{4}u^3, & |u| < 1, \\ 1, & u \geq 1, \end{cases} \quad K'(u) = \frac{3}{4}(1 - u^2) \mathbf{1}[|u| < 1].$$

Blocks more than h apart in score contribute *exactly* 0 or *exactly* τ_i . Only genuinely near-tied blocks are smoothed, which is the only place smoothing was ever wanted. K is C^1 ; use the biweight (quintic CDF) if C^2 is needed. This also matches the Wendland compact-support instinct already used elsewhere in the codebase — though note it is a kernel on the *score* line, unrelated to the geometric kernel in `anchor_interpolation.py`.

(C) Emit t directly and skip the rank. Tempting, but it does not work. Given an arbitrary head output t , the schedule it implies is the cumulative sum of τ *in t 's induced order*, which is not t itself; enforcing consistency projects t onto $\{\text{cumsum of } \tau \text{ under a permutation}\}$, a discrete set. The rank is relocated, not removed. Recording this so we do not re-derive it later.

6. Cost, and an honest note on the anchor interpolation

With compact support the smoothed G_h is computable *exactly* and cheaply. Sort the scores once; then

$$G_h(s_j^*) = \underbrace{\sum_{i: s_i^* \geq s_j^* + h} \tau_i}_{\text{prefix sum, } O(1)} + \underbrace{\sum_{i: |s_i^* - s_j^*| < h} \tau_i K\left(\frac{s_i^* - s_j^*}{h}\right)}_{\text{local window, } O(k)},$$

giving $O(n \log n + nk)$ with k the mean window occupancy. The sort is a permutation, so gradients flow through the gather unimpeded — we never differentiate the argsort, only K .

Remark 2 (the anchor/knot interpolation is not needed here). In the previous note I proposed evaluating G_h at m quantile knots and interpolating. Given the prefix-sum construction above

that is *slower and less exact*, not faster. Knots earn their place only if h is large enough that $k \sim n$, or if G must be reused across a batch without re-sorting. Answering the original question directly: yes, kernel interpolation *can* build t from the projected scores — but it should not, because sorting already does it exactly.

7. What is still open

1. **Horizon.** As written every block is eventually mined and merely discounted. If blocks past period T are instead *abandoned*, that is a genuine discontinuity in value, not in time, and needs its own decision — it is not the same thing as the horizon gate we just deleted. Deferring until you say which.
2. **Bandwidth h .** Set it as a score-quantile spacing so it is scale-free as the score distribution drifts, and anneal toward 0; at $h \rightarrow 0$ the surrogate becomes the exact schedule.
3. **Tie-break.** Still needed: exact ties give equal start times. Split-mass $(+\frac{1}{2} \sum_i \tau_i \mathbf{1}[s_i^* = s_j^*])$ is the smooth choice and is what the kernel does automatically at $u = 0$, since $K(0) = \frac{1}{2}$. So with a kernel the tie-break is already handled; only the hard path (A) needs an explicit rule.
4. **Validation.** Report exact NPV from the hard sort-cumsum against the smoothed objective at the chosen h . That gap is now the *only* surrogate error in the pipeline.